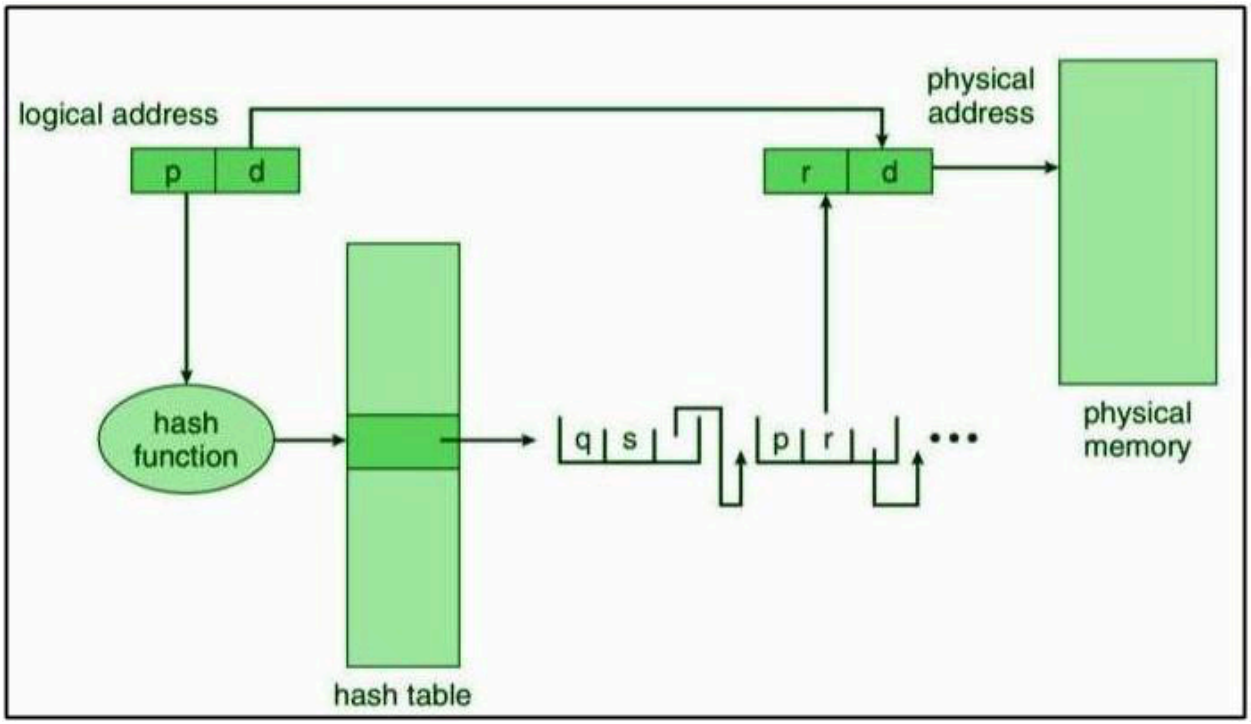
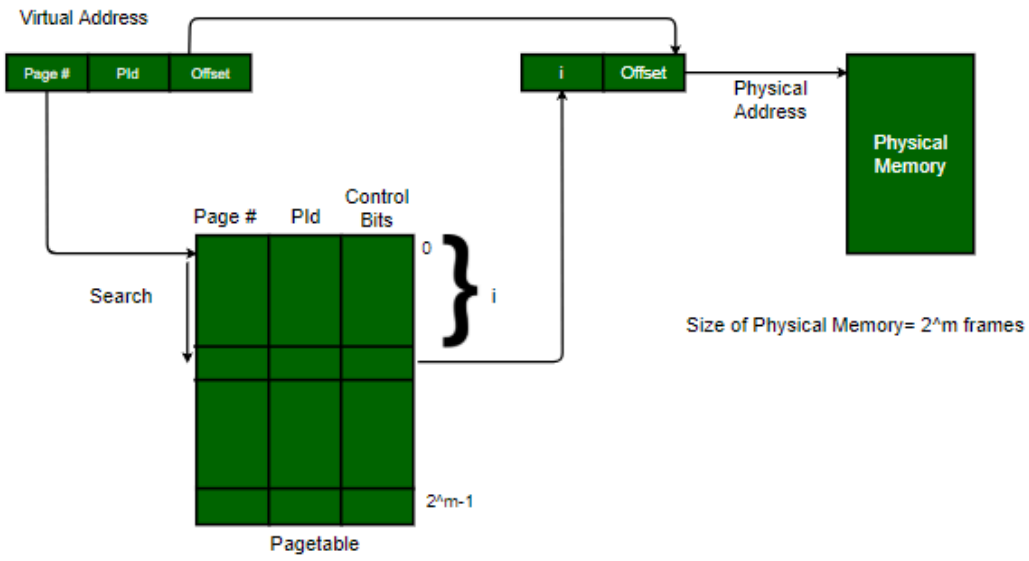
**Hashed Page Tables and Inverted Page Tables:**

Page tables are one of the most important parts of modern memory management systems as they provide a mechanism that translates a virtual address to a physical address. Since virtual address space has increased over time, there are several different designs for page tables. The Hashed Page Table (HPT) and Inverted Page Table (IPT) are two of the most important ones. Despite both page tables working to increase space efficiency and improve address translation, they mapped addresses in completely different ways with different performance characteristics.




**Hashed Page Tables**




**Inverted Page Table**

**Memory Organization and Structure:**

Hashed Page Table (HPT)

A hashed page table is ideal for large or sparsely filled virtual address spaces like for 64-bit architectures. The hashed page table design uses a hash function to translate a virtual page number into an index in the hash table. Each hash bucket contains a linked list of entries and each entry contains a virtual page number. Along with that, it also holds the corresponding physical frame number. Finally, there is a pointer that connects the entry to the next one in the chain. The OS computes the hash to translate the virtual address, traverses the linked list until it finds the same virtual page number and lastly combines the physical frame number with the page offset to generate the physical address.

Hashed page tables are not without their limitations. For example, hashed page tables only store mappings for pages that are currently in memory. And thus reduce memory overhead and provide flexibility in the case of sparse address spaces. Generally, hashed page tables incur lower overhead compared to flat page tables or hierarchical page tables but slightly more overhead compared to inverted page tables. Because each process maintains its own page mappings. Also hash collisions can exist when there are many virtual page numbers mapped to the same bucket hash, requiring linked list traversal, which has an expected average lookup time of $O(1 + \alpha)$. Here $\alpha$ is the average number of entries corresponding to the hashed bucket.

Inverted Page Table (IPT)

The Inverted Page Table (IPT) organizes memory differently than conventional or hashed page tables. The IPT has a single global table. It can have as many entries as physical frames for all processes. Each entry contains a virtual page number, an associated process ID and a collection of control bits such as protection bits or dirty bits. This organization allows memory overhead to be dependent on the size of physical memory rather than the size of the virtual address space, which is very space efficient.

The process of translating an address in an IPT is more complex than in hashed page tables. To translate a virtual address using an IPT, the OS combines the process ID and virtual page number to mapped entry. If a hash function is used to create the mapping, collisions will be resolved in chains or through probing. Otherwise a linear search must be performed. After the correct entry is retrieved, the physical frame number can be combined with the page offset to create a physical address. Overall, IPTs result in memory that is compact yet mapping to a physical address may be slightly slower and more complex than using a conventional or hashed page table.

**Efficiency Comparison**

| Aspect | Hashed Page Table (HPT) | Inverted Page Table (IPT) | Traditional Page Table |
|---|---|---|---|
| **Memory Overhead** | **Moderate (per-process entries for active pages)** | **Low (size depends on physical frames only)** | **High (entries for all virtual pages)** |
| **Lookup Speed** | **Fast (hashing reduces search time)** | **Slower (may require hash collision resolution or linear search)** | **Moderate (direct indexing if small, slower if large)** |
| **Flexibility for Sparse Address Spaces** | **High (stores only pages in use)** | **Moderate (single table for all processes)** | **Low (must allocate entries for all virtual pages)** |
| **Scalability** | **Good (handles large address spaces well)** | **Very Good (efficient for physical-memory-limited systems)** | **Limited (becomes inefficient with large address spaces)** |

Hashed page tables and inverted page tables exhibit different trade-offs with respect to memory efficiency and lookup complexity. HPTs are more flexible and have a higher lookup speed for typical sparse addresses spaces. And IPTs reduce the expected memory overhead by using one single, compact table size for all processes. Understanding these trade-offs is useful in designing virtual memory systems.

**Performance Characteristics:**

**Hash Collisions and Search Overhead**

With a hashed page table, the primary issue is hash collisions. Each virtual page number hashes to a bucket but sometimes multiple pages can be hashed to the same location. If multiple page entries hashed to the same location, the system must read down a linked list in that bucket to select the correct entry. In the best case, a mapping may be reachable on the first attempt and in the worst case, it may take searching through several entries. However, even with collisions, hashed page tables tend to be quicker than inspecting a full traditional page table, particularly with large or sparse virtual address spaces.

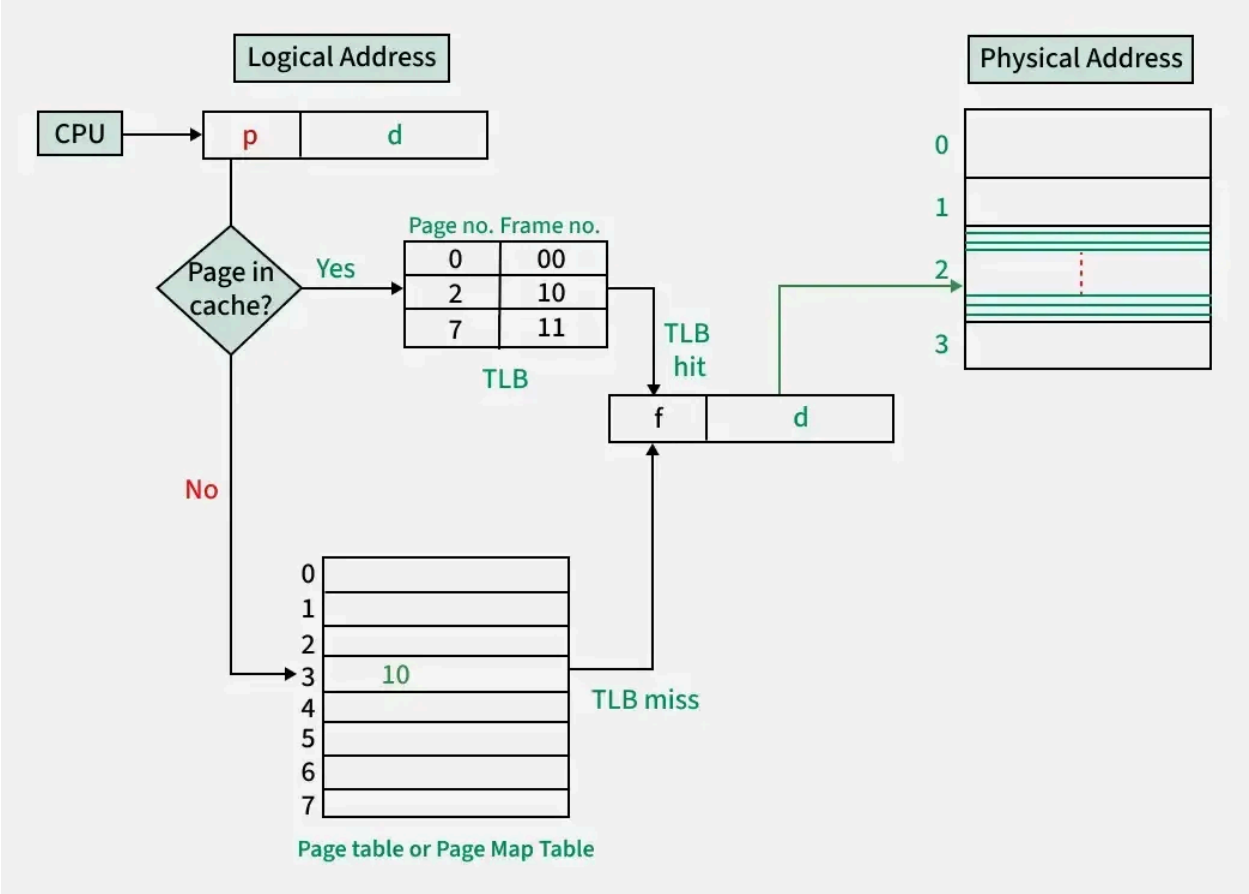
The inverted page table does not have this degree of simplicity. Each process can have the same virtual page number but different process identifiers. And the inverted page table is a single global table for all processes. Each entry in the inverted page table holds the virtual page number, process identifier and control bits. Searching for a virtual page number requires knowing the process identifier. So already this adds complexity over a hashed page table. Collisions will also lengthen the time for each search. Dealing with collisions is likely to add time to the address translation process, particularly in systems with numerous active processes.

**TLB Miss Handling and Page Faults**

Both hashed and inverted page tables utilize the Translation Lookaside Buffer (TLB) to enable faster accesses to memory. A TLB hit indicates the translation can be executed immediately. A TLB miss indicates that the page table needs to be looked up.

For the hashed page table, the processor calculates the hash, looks through a linked list in that bucket. And finds the frame number. If the list doesn't contain the entry for that page number, a page fault occurs and the secondary storage loads the missing page.

For an inverted page table, the combination of virtual page number and process ID is hashed, which is then looked up in the global table with the collision resolution. If the page is not present in memory, a page fault occurs. However, not only does the software need to bring in the missing page, the global table may need to be updated with the new process ID. This may add additional overhead.




**Memory vs. Speed Trade-Off**

In terms of memory consumption, hashed page tables use more memory than inverted page tables because each process uses its own table. They allow faster lookups, particularly in sparse address spaces. Inverted page tables are generally more memory efficient. It is due to the table's size being determined by the number of physical frames instead of each process creating its own table. However, because inverted page tables use a global search, translation can take longer and may incur time related to handling collisions.

Overall, hashed page tables work well in situations where the systems use large, sparse virtual address spaces. As it would be the case with 64-bit address spaces. On the other hand, inverted page tables work better when memory is constrained and saving memory is more important than lookup speed.

In other words, the deferred cost of additional memory consumption of hashed page tables is more than compensated by the faster lookups compared to the speeding system while using inverted page tables. It boils down to a trade off between lookup speed and memory efficiency. Hashed page tables will allow for quicker translations than inverted page tables will consume more memory but use less time or memory rather than trying to minimize lookup speed by limiting overhead during TLB misses and page faults.

**Conclusion:**

In conclusion, hashed page tables and inverted page tables are two different methods for translating virtual addresses into physical addresses. Hashed page tables are flexible and efficient for dealing with large and sparse virtual address spaces because they use hash functions to fit a table in a smaller size and deal with hash collisions. Inverted page tables are efficient in space safety. They have no per-process table and they only keep one entry per physical frame. But the translation step is more complicated and the additional translation step will introduce more overhead in relation to page faults. The choice about which method to use should be dictated by the system's priorities which can be speed of access and ability to handle sparse memory (hashed page tables) or fastest space efficiency (inverted page table). Each design is indicative of the trade-offs, speed, memory efficiency, complexity that exist inside of all modern virtual memory management systems.

**References:**


1. https://www.geeksforgeeks.org/operating-systems/difference-between-page-table-and-inverted-page-table/
2. https://www.geeksforgeeks.org/operating-systems/hashed-page-tables-in-operating-system/
3. https://medium.com/@parul947a/memory-management-in-os-3d385c66438b
4. https://medium.com/@souravganguly_32007/inverted-paging-was-a-concept-that-was-introduced-after-the-conventional-paging-system-in-order-575ced00bda6
5. https://medium.com/codesphere-cloud/hash-tables-how-they-work-and-why-they-matter-683a464f486
6. https://www.geeksforgeeks.org/operating-systems/translation-lookaside-buffer-tlb-in-paging/